

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: data = pd.read_csv("C:/Users/System05/Downloads/heart.csv")
```

```
In [3]: data.head()
```

```
Out[3]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	1

```
In [4]: data.isnull().sum()
```

```
Out[4]: age          0
sex            0
cp             0
trestbps       0
chol           0
fbs            0
restecg        0
thalach        0
exang          0
oldpeak        0
slope          0
ca             0
thal           0
target         0
dtype: int64
```

```
In [5]: X = data.drop("target", axis=1)
y = data["target"]
```

```
In [6]: from sklearn.model_selection import train_test_split
```

```
In [7]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
In [8]: from sklearn.linear_model import LogisticRegression
```

```
In [9]: model = LogisticRegression(max_iter=1000)
```

```
In [10]: model.fit(X_train, y_train)
```

C:\Users\System05\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:469: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(

Out[10]:

LogisticRegression
LogisticRegression(max_iter=1000)

In [11]: y_pred = model.predict(X_test)

In [12]: print("Predicted values:")
print(y_pred)

Predicted values:

```
[0 1 1 1 0 1 1 1 1 0 1 0 1 0 0 1 0 1 0 0 1 0 0 0 1 1 0 1 0 1 1 0 0 1 1 1 1
 0 0 1 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 1 0 0 1 0 0 0 1 1 0 0 1 1 1 0 0 1
 1 1 1 0 1 1 0 1 1 1 0 0 1 0 1 0 1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 1 0 1 1 1 0
 1 0 1 1 1 0 1 1 1 0 0 0 0 1 0 1 1 1 1 0 0 0 0 1 1 1 0 1 1 0 0 0 1 0 1 1 1
 0 1 1 1 0 0 0 1 0 1 1 0 1 1 1 1 0 0 1 1 0 1 1 0 1 1 1 1 1 0 0 0 1 1 0 0
 0 1 0 1 1 1 1 0 0 0 1 1 0 0 1 1 0 0 1 0 0 1 0 0 1 1 0 0 1 0 1 1 1 1 1 0 1
 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 0 1 0 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 0
 0 0 0 0 0 0 1 1 0 1 0 0 0 0 1 1 0 1 1 1 1 0 0 1 0 0 1 1 0 1 0 1 0 1 0 0 0
 0 1 1 0 1 1 1 1 0 1 1 1]
```

In [13]: from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

In [14]: accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

Accuracy: 0.8116883116883117

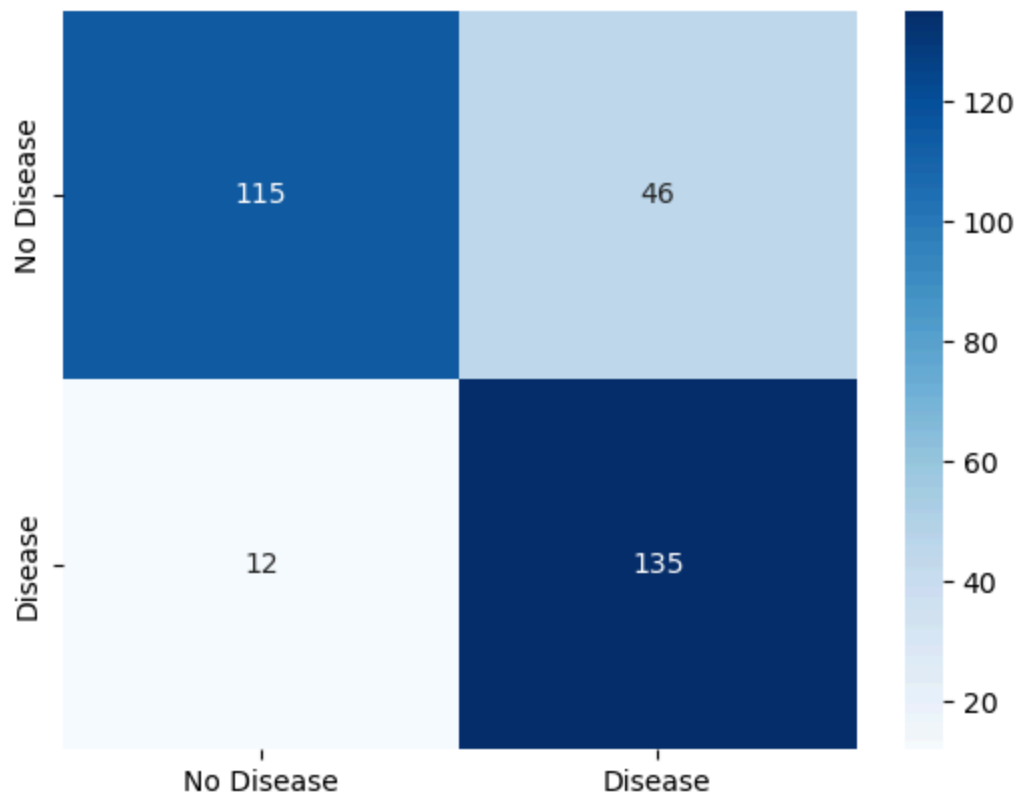
In [15]: cm = confusion_matrix(y_test, y_pred)
print(cm)

```
[[115  46]
 [ 12 135]]
```

In [16]: import seaborn as sns

In [17]: sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=["No Disease", "Disease"],
yticklabels=["No Disease", "Disease"])

Out[17]: <Axes: >



In []: